

Documentație

Titlu

Aethelgard: Rise of the Fallen Crown

Gen

Joc 2D tile-based, top-down adventure, realizat în Java, cu elemente fantasy, explorare, coliziuni, inamici simpli, tranziții între niveluri și interfață grafică proprie.

Poveste

Jucătorul controlează un cavaler trimis să recupereze coroana pierdută a regatului Aethelgard. Regatul a fost corupt de o forță întunecată, iar eroul trebuie să treacă prin trei zone periculoase, să evite sau să înfrunte inamici, să găsească obiecte-cheie și să ajungă la confruntarea finală. În versiunea curentă, jocul pune accent pe explorarea hărților, mișcarea personajului, evitarea obstacolelor și parcurgerea celor trei niveluri implementate.

Pe parcursul dezvoltării, povestea va fi extinsă prin introducerea unor NPC-uri, obiecte-cheie, iteme colectabile, scor, salvare/încărcare și o confruntare finală mai bine definită cu antagonistul principal.

Obiectiv

Obiectivul principal al jocului este parcurgerea celor trei niveluri, depășirea obstacolelor și ajungerea la confruntarea finală, în urma căreia jucătorul recuperează coroana regatului. În etapa actuală, progresia se face prin atingerea zonelor de ieșire din niveluri, care declanșează trecerea către nivelul următor. În etapele următoare, obiectivul va fi completat cu iteme necesare progresului, scor, save/load și finalizarea narativă a jocului.

Mecanici implementate până în etapa curentă

- deplasarea personajului principal cu tastatura;
- control prin tastele W/A/S/D și săgeți;
- încărcarea hărților tile-based din fișiere externe `.txt`;
- trei niveluri funcționale: pădure, dungeon și sală finală;
- coliziuni cu tile-uri solide, precum apă, copaci, ziduri, munți, butoaie sau cufere;
- sistem de hitbox pentru personaj și inamici;
- cameră care urmărește jucătorul și este limitată la marginile hărții;
- animații pentru cavaler: idle, run și turn around;
- inamic de tip lup în primul nivel;
- inamic de tip schelet în al doilea nivel;
- tranziții între niveluri;
- meniu principal grafic;
- ecran de setări;
- meniu de pauză accesibil cu Escape;
- opțiune de revenire în meniul principal din timpul jocului;
- opțiune de ieșire din aplicație;
- mod debug pentru afișarea hitbox-urilor și a FPS-ului.

Mecanici planificate pentru etapele următoare

- NPC-uri și dialoguri;
- iteme colectabile;
- scor și/sau XP;
- sistem de save/load;
- salvarea progresului într-o bază de date;
- clasament pentru scoruri;
- excepții personalizate;
- boss sau eveniment final în nivelul 3;
- extinderea sistemului de inamici;
- finisare grafică și optimizare.

Nivelul 1 — The Cursed Forest

- hartă exterioară
- copaci, pietre, drumuri blocate
- inamici simpli
- obiectiv: găsește cheia ruinelor (obiectiv narativ propus)

Nivelul 2 — Beneath the Castle

- hartă interioară
- coridoare, capcane, uși
- NPC / spirit guardian (obiectiv narativ propus)
- obiectiv: găsește artefactul care deschide sala finală (obiectiv narativ propus)

Nivelul 3 — The Great Hall

- nivel final
- confruntare cu antagonistul (obiectiv narativ propus)
- obiectiv: recuperează coroana și finalizează jocul (obiectiv narativ propus)

1.Contextul jocului

Aethelgard: Rise of the Fallen Crown este un joc 2D de tip top-down adventure, bazat pe un sistem tile-based. Acțiunea se desfășoară într-un regat fantasy aflat în ruină, corupt de o forță întunecată. Jucătorul controlează ultimul cavaler loial coroanei și trebuie să exploreze zone periculoase, să depășească obstacole, să interacționeze cu personaje și să recupereze coroana pierdută. Jocul este proiectat să combine explorarea, progresia între niveluri, sistemul de scor și mecanica de salvare/încărcare a progresului. În versiunea curentă, sunt implementate explorarea, progresia între niveluri, camera, coliziunile și inamicii simpli.

De asemenea, proiectul are deja o structură funcțională de joc: meniu principal, ecran de setări, fereastră de joc, buclă principală de update și draw, hartă tile-based, personaj controlabil, cameră, coliziuni, inamici simpli și tranziții între niveluri.

Jocul este construit gradual, conform etapelor de laborator. În această etapă, accentul a fost pus pe partea de bază a gameplay-ului: afișarea hărților, controlul personajului, animațiile, coliziunile, camera, nivelurile multiple și interfața grafică. Funcționalitățile de scor, salvare/încărcare și bază de date urmează să fie integrate în etapele următoare.

2. Proiectarea sistemului / gameplay

Jocul pornește din clasa Main, care creează o instanță a meniului principal. Meniul principal este implementat prin clasa MenuWindow și permite utilizatorului să pornească un joc nou, să acceseze ecranul de setări, să închidă aplicația sau să observe opțiunea de Load Game, care urmează să fie implementată în etapa dedicată salvării și încărcării progresului.

După apăsarea butonului Play, este creată o instanță a clasei Game, care pornește bucla principală de joc într-un fir separat de execuție. Clasa Game este responsabilă pentru inițializarea resurselor, încărcarea hărții, actualizarea stării jocului, verificarea tranzițiilor dintre niveluri și randarea elementelor grafice.

Gameplay-ul este organizat în jurul unei bucle principale de tip *game loop*. La fiecare frame, jocul:

1. citește inputul de la tastatură;
2. actualizează starea jucătorului;
3. actualizează inamicii existenți în nivelul curent;
4. actualizează poziția camerei;
5. verifică dacă jucătorul a ajuns într-o zonă de tranziție;
6. redesenează harta, inamicii, player-ul și eventualele overlay-uri.

Hărțile sunt încărcate din fișiere externe, aflate în folderul res/maps, de exemplu level1.txt, level2.txt și level3.txt. Fiecare hartă este reprezentată printr-o matrice de numere, unde fiecare număr corespunde unui tip de tile. Astfel, harta nu este creată direct în cod, ci este citită din fișier și transformată într-o structură internă de tip matrice.

Personajul este controlat de la tastatură, folosind tastele W/A/S/D sau săgețile. Clasa KeyManager gestionează starea tastelor și oferă clasei Game informații despre direcția de deplasare, apăsarea tastei Escape, Enter sau activarea modului debug.

3. Șabloane de proiectare utilizate

Game Loop

Deși nu este un șablon GoF clasic, *Game Loop* este un model arhitectural fundamental pentru jocuri. În proiect, acesta este implementat în clasa Game, prin metoda run(). Bucla principală rulează cât timp jocul este activ și controlează actualizarea și randarea scenei la aproximativ 60 FPS.

Acest model permite separarea clară între:

- logica jocului, realizată în metoda Update();
- randarea grafică, realizată în metoda Draw();
- controlul timpului, prin limitarea numărului de cadre pe secundă.

Flyweight pentru tile-uri

Sistemul de tile-uri folosește o abordare apropiată de șablonul *Flyweight*. În loc să creeze câte un obiect nou pentru fiecare celulă a hărții, proiectul definește tipurile de tile ca instanțe statice în clasa Tile. Harta păstrează doar ID-urile tile-urilor, iar la randare se recuperează referința corespunzătoare din vectorul static Tile.tiles.

Această abordare reduce consumul de memorie, deoarece același obiect de tip tile este refolosit în mai multe poziții ale hărții. De exemplu, toate celulele de iarbă folosesc aceeași instanță grassTile, toate celulele de apă folosesc aceeași instanță waterTile, iar toate zidurile de dungeon folosesc aceeași instanță dungeonWallTile.

Resource Manager / Singleton simplificat

Clasa *Assets* funcționează ca un manager global de resurse grafice. Aceasta încarcă imaginile necesare jocului și le pune la dispoziția celorlalte clase prin câmpuri statice. În acest fel, resursele grafice sunt încărcate centralizat, nu separat în fiecare clasă care are nevoie de ele. Această abordare poate fi considerată o variantă simplificată de *Resource Manager* cu comportament asemănător unui Singleton, deoarece există un singur punct central prin care sunt încărcate resursele principale ale jocului.

Moștenire și polimorfism pentru entități

Entitățile principale din joc pornesc de la o clasă de bază comună, *Entity*. Aceasta conține poziția, dimensiunea, viteza și logica generală de coliziune. Clasele *Player*, *Enemy* și *Skeleton* extind această bază și adaugă comportamente specifice.

Această structură permite reutilizarea logicii comune și extinderea mai ușoară a jocului cu alte entități, precum NPC-uri, iteme interactive sau noi tipuri de inamici.

Extensii planificate

Pentru etapele următoare, proiectul poate fi extins prin:

- un *Factory* pentru crearea inamicilor;
- un *State Pattern* pentru stările jocului: *Main Menu*, *Playing*, *Paused*, *Game Over*, *Victory*;
- un sistem de excepții personalizate pentru încărcarea resurselor, hărților și salvărilor.

4. Proiectarea interfeței cu utilizatorul

Interfața jocului este realizată în Java folosind componente Swing/AWT și desenare custom prin *Graphics2D*.

Meniul principal

Meniul principal este implementat în clasa *MenuWindow*. Acesta are o atmosferă dark fantasy, potrivită temei jocului. Fundalul este desenat grafic și include elemente precum cer întunecat, munți, lună și titlul jocului.

Meniul principal conține următoarele opțiuni:

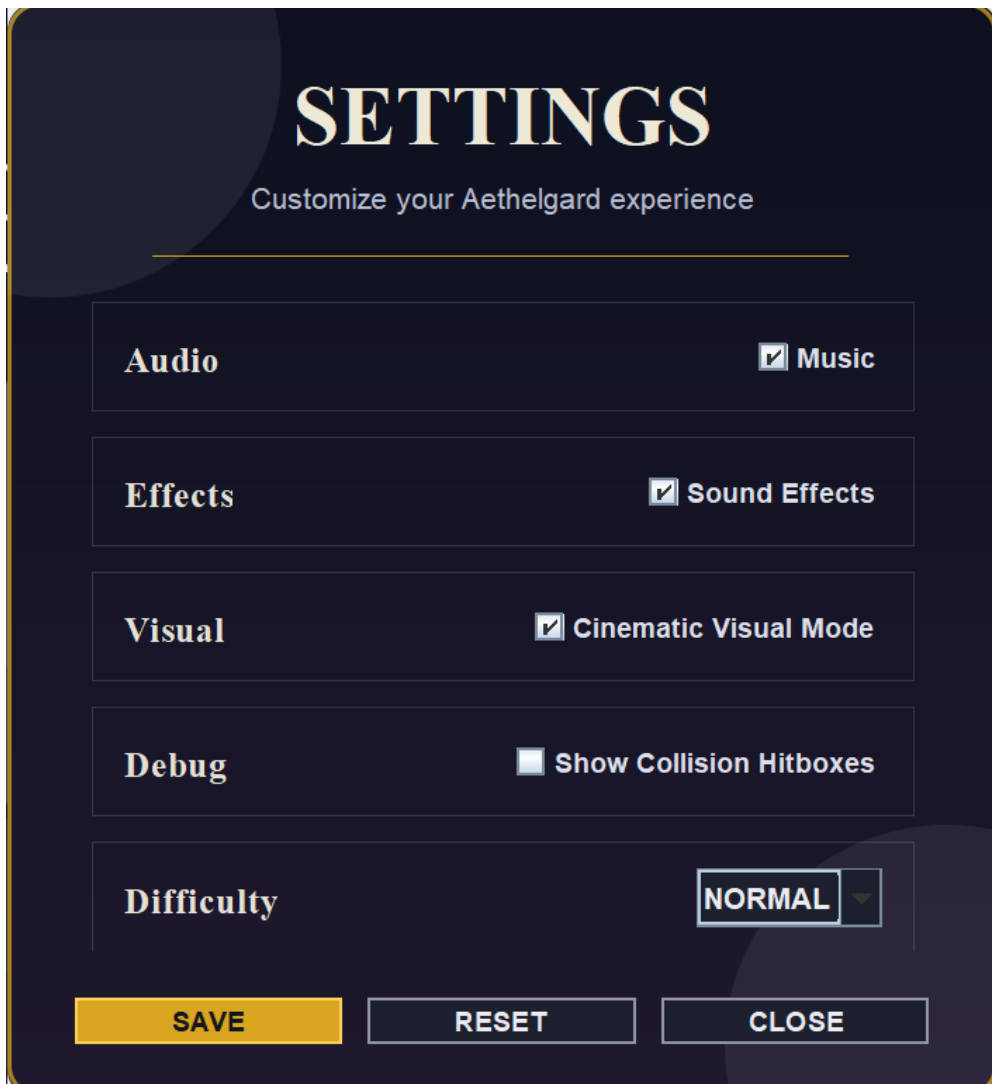
- ***Play*** — pornește jocul;
- ***Load Game*** — opțiune pregătită pentru etapa de save/load;
- ***Settings*** — deschide ecranul de setări;
- ***Exit*** — închide aplicația.

Ecranul de setări

Ecranul de setări este implementat prin clasa *SettingsDialog*. Acesta permite modificarea unor opțiuni globale ale jocului:

- Music;
- Sound Effects;
- Cinematic Visual Mode;
- Show Collision Hitboxes;
- Difficulty: EASY, NORMAL, HARD.

Setările sunt păstrate în clasa *GameSettings*. În etapa actuală, acestea sunt salvate doar în memorie, iar în etapa de bază de date pot fi persistate.

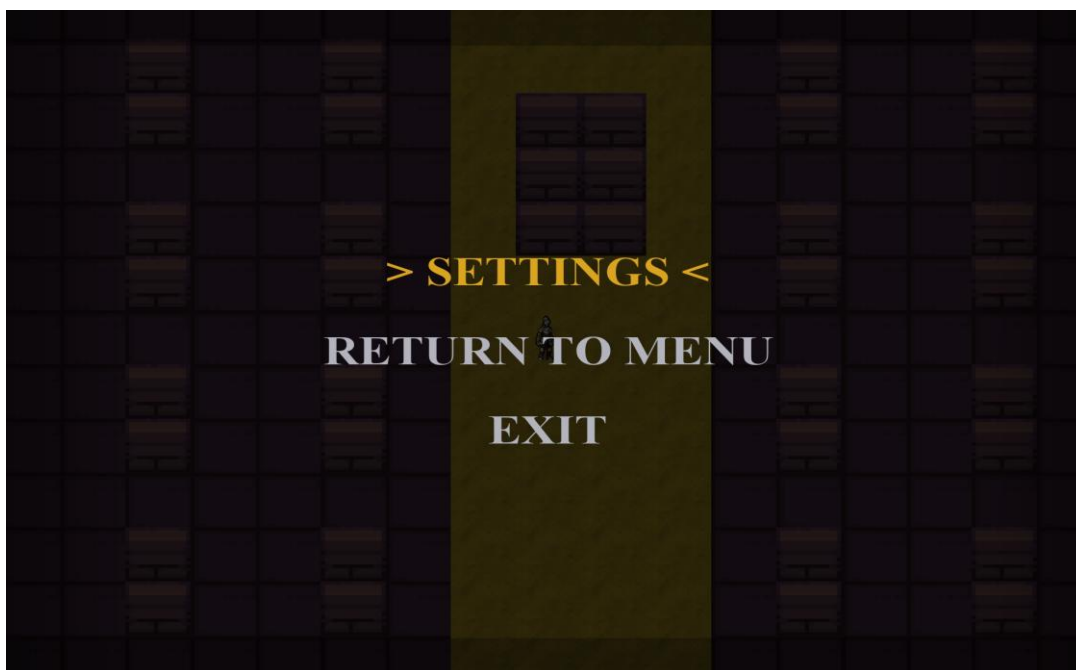


Meniul de pauză

În timpul jocului, apăsarea tastei **Escape** activează meniul de pauză. Acesta este desenat ca overlay semi-transparent peste joc și conține trei opțiuni:

- ***Settings;***
- ***Return to Menu;***
- ***Exit.***

Opțiunea *Return to Menu* oprește bucla jocului, închide fereastra curentă și recrează meniul principal. Această soluție previne problemele grafice apărute la revenirea din joc în meniu și asigură reinițializarea corectă a ferestrei.



Pentru susținerea proiectării interfeței, au fost realizate mockup-uri conceptuale pentru meniul principal și pentru ecranul de setări/controale ilustrate în figurile de mai sus.

5. Proiectarea conținutului

Conținutul jocului este construit în jurul unei lumi dark fantasy și al unui parcurs narativ împărțit în trei capitole. Personajul principal este ultimul cavaler loial coroanei regatului Aethelgard, trimis să recupereze coroana pierdută și să restabilească ordinea într-un regat corupt de magie întunecată. Antagonistul principal este Malakar, un vrăjitor care a adus regatul în ruină și care reprezintă obstacolul final al poveștii. Structura capitolelor și confruntarea finală cu Malakar apar și în materialele auxiliare, ceea ce face alegerea foarte potrivită pentru proiect.

În joc vor exista mai multe tipuri de entități interactive, modelate printr-o superclasă comună Entity. Personajul principal (Player) va fi controlat de utilizator și va putea explora hărțile, colecta obiecte și interacționa cu alte personaje. Inamicii (Enemy) vor fi de tipul Corrupted Guard și Shadow Beast, iar rolul lor va fi să blocheze progresul jucătorului și să introducă dificultate pe parcursul nivelurilor. În plus, va exista cel puțin un NPC important (NPC), de exemplu Spirit Guardian sau Imprisoned Wizard, care va oferi informații, va ghida jucătorul sau va influența progresul poveștii.

Obiectele principale ale jocului vor include cheia ruinelor, artefactul necesar pentru accesul în zona finală, coroana pierdută și obiecte consumabile precum potion. Aceste iteme vor avea rol funcțional în progresia dintre niveluri și pot contribui la sistemul de scor și la deblocarea unor căi alternative. Pentru a face jocul mai interesant, conținutul va include și un sistem de progresie bazat pe XP și points, precum și realizări speciale de tip achievements și

endings. Exemple potrivite sunt: *First Blood*, *Into the Woods*, *The Forgotten Cell*, *Master of Mercy*, precum și ending-urile *Crown Reclaimed* sau *Exile of Malakar*.

Conținutul implementat până în etapa curentă include personajul principal, trei hărți tile-based și doi inamici simpli.

Personajul principal este un cavaler animat, controlat de utilizator. Acesta are animații pentru:

- starea de repaus;
- deplasare;
- întoarcere / schimbare de direcție.

Inamicii implementați sunt:

- un lup în primul nivel;
- un schelet în al doilea nivel.

Atât lupul, cât și scheletul folosesc o logică simplă de urmărire a jucătorului. Fiecare inamic are o rază de detecție, iar atunci când jucătorul intră în această rază, inamicul încearcă să se apropie de el. Mișcarea inamicilor este verificată prin același sistem de coliziune cu harta, astfel încât aceștia să nu treacă prin tile-uri solide.

În etapa actuală, conținutul narativ este încă la nivel de structură generală. NPC-urile, dialogurile, itemele colectabile și sistemul de scor urmează să fie integrate în etapele următoare.

6. Proiectarea nivelurilor

Jocul conține trei niveluri, fiecare încărcat dintr-un fișier text extern. Fiecare hartă este construită din tile-uri, iar fiecare tile are un ID corespunzător în matricea hărții.

Nivelul 1 — The Cursed Forest

Primul nivel reprezintă o zonă exterioară, o pădure blestemată de la marginea regatului. Harta folosește tile-uri precum iarbă, sol, apă, copaci și munți/piatră.

Rolul acestui nivel este de a introduce jucătorul în mecanicile de bază:

- deplasare;
- coliziuni;
- explorare;
- urmărire de către un inamic simplu.

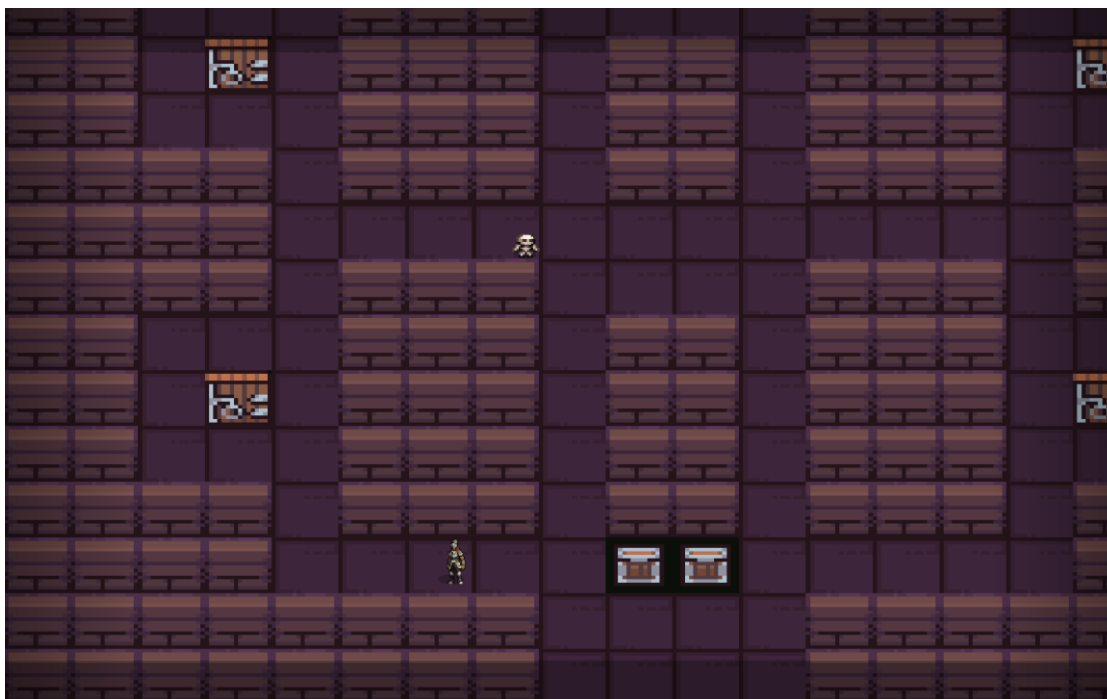
În acest nivel apare inamicul de tip lup. Lupul urmărește jucătorul atunci când acesta intră în raza lui de detecție. Pentru a ajunge în nivelul următor, jucătorul trebuie să ajungă în zona de ieșire din partea superioară a hărții.



Nivelul 2 — Beneath the Castle

Al doilea nivel este o zonă de dungeon, cu atmosferă interioară. Harta folosește tile-uri precum podea de dungeon, ziduri, butoaie și cufere. Spre deosebire de primul nivel, spațiul este mai restrâns, iar traseul este mai apropiat de structura unor coridoare.

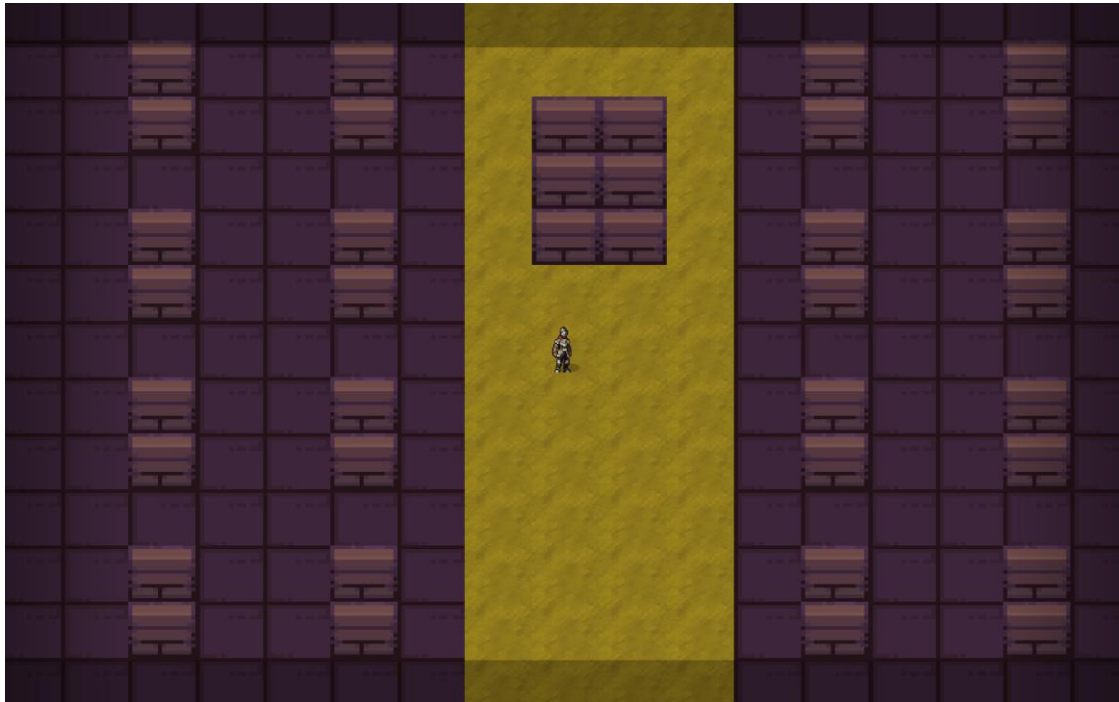
În acest nivel apare un inamic de tip schelet. Scheletul are animații proprii de idle și mers și urmărește jucătorul într-o rază limitată. Obiectivul nivelului este atingerea zonei de ieșire, care declanșează tranziția către nivelul 3.



Nivelul 3 — The Great Hall

Al treilea nivel reprezintă zona finală a jocului. În etapa actuală, acest nivel este încărcat și accesibil prin sistemul de tranziție, funcționând ca hartă finală de bază. El urmează să fie

completat cu elemente de finalizare narativă, antagonist principal, scor și eventual un obiectiv final.



7. Sistemul de coliziuni

Sistemul de coliziuni este implementat în clasa Entity, prin metoda CanMoveTo(). Fiecare entitate are o poziție, o dimensiune și un hitbox efectiv folosit pentru coliziuni.

În loc să se folosească întreaga imagine a personajului pentru coliziuni, proiectul folosește un hitbox mai mic, poziționat în zona picioarelor. Această alegere este potrivită pentru un joc top-down, deoarece permite ca sprite-ul să se suprapună vizual peste unele elemente, fără ca jucătorul să pară blocat incorect.

Pentru verificarea coliziunii, se testează cele patru colțuri ale hitbox-ului. Dacă unul dintre aceste puncte ajunge pe un tile solid, mișcarea este blocată.

Tile-urile solide sunt definite prin suprascrierea metodei IsSolid(). De exemplu, apa, copacii, munții, zidurile de dungeon, butoaiile și cuferele sunt tratate ca obstacole prin care jucătorul nu poate trece.

Modul debug permite afișarea hitbox-urilor pe ecran. Tile-urile solide sunt evidențiate, iar pentru entități se afișează atât bounding box-ul general, cât și hitbox-ul real folosit pentru coliziuni.

8. Camera

Camera este implementată prin clasa Camera. Rolul ei este să urmărească poziția jucătorului și să afișeze doar porțiunea relevantă din hartă.

La fiecare update, camera se centrează pe jucător. Pentru a preveni afișarea unor zone în afara hărții, poziția camerei este limitată între marginile hărții. Astfel, atunci când jucătorul se apropie de marginea nivelului, camera nu continuă să se deplaseze în afara spațiului valid.

În timpul randării, coordonatele lumii sunt transformate în coordonate de ecran prin scăderea poziției camerei. Astfel, harta, inamicii și jucătorul sunt desenate relativ la poziția curentă a camerei.

Pentru hărțile mai mici decât fereastra logică, jocul calculează un offset de centrare, astfel încât harta să nu apară lipită de marginea ferestrei.

9. Sprite-uri și resurse grafice

Proiectul va utiliza minimum două game sprite-uri, conform cerinței. Primul sprite va reprezenta personajul principal, adică cavalerul controlat de utilizator, iar al doilea va reprezenta un inamic al jocului, de exemplu un lup ostil (Wolf Enemy). În plus, vor fi utilizate tile-uri pentru teren și decor, precum grass, stone, wall, water și dungeon floor. Resursele grafice pot fi obținute din biblioteci specializate de asset-uri sau realizate/adaptate în editoare dedicate precum Aseprite, GIMP sau alte unelte recomandate în suportul de curs.

Proiectul utilizează minimum două game sprite-uri, conform cerinței. În versiunea curentă sunt folosite trei sprite-uri principale: cavalerul, lupul și scheletul.

Sprite 1: Knight Player Sprite



Sprite 2: Wolf Enemy Sprite



Sprite 3: Skeleton



Cavalerul folosește sprite sheet-uri separate pentru animațiile de idle, run și turn around. Fiecare animație este împărțită în cadre, iar clasa Animation gestionează trecerea de la un cadru la altul. Lupul este încărcat dintr-un sprite sheet mai mare, din care sunt decupate cadrele pentru idle și run. Scheletul folosește sprite sheet-uri separate pentru idle și mers. Pentru tile-uri, proiectul folosește atât sprite sheet-ul principal al jocului, cât și un tileset de dungeon. Din acestea sunt decupate imaginile pentru iarbă, sol, apă, copaci, munți, podea de dungeon, ziduri, butoaie și cufere.

10. Sistem de progresie, scor și save/load

În etapa actuală, jocul se concentrează pe mecanicile de bază: mișcare, coliziuni, cameră, niveluri, inamici și interfață. Sistemul complet de scor, XP, salvare și încărcare urmează să fie implementat în etapele următoare.

Funcționalitatea de Load Game este deja prezentă în meniul principal ca element de interfață, însă în această versiune afișează un mesaj temporar, urmând să fie conectată ulterior la baza de date.

Pentru etapa de save/load se vor salva cel puțin următoarele informații:

- nivelul curent;
- poziția jucătorului;
- scorul;
- timpul sau progresul;
- eventualele iteme colectate;
- starea jocului la momentul salvării.

Baza de date va permite încărcarea unui progres anterior și păstrarea celor mai bune scoruri, conform cerințelor proiectului.

11. Tratarea excepțiilor

În etapa curentă, tratarea erorilor este realizată în principal prin blocuri try-catch, folosite la încărcarea hărților și a imaginilor. Dacă o resursă nu poate fi încărcată, eroarea este afișată în consolă.

În etapa de finisare, sistemul poate fi extins prin excepții personalizate, precum:

- ResourceLoadException;
- MapLoadException;
- SpriteNotFoundException;
- InvalidLevelException;
- DatabaseConnectionException;
- InvalidSaveException.

Aceste clase vor permite separarea mai clară a erorilor și tratarea lor într-un mod mai robust.

12. Sarcinile săptămânale ale membrilor echipei

Planificarea sarcinilor a fost realizată în conformitate cu cerințele oficiale ale proiectului. Task-urile vor fi urmărite în GitHub Projects și corelate cu commit-urile din repository. Începând cu săptămâna 8, fiecare membru al echipei va realiza cel puțin un commit pe săptămână, conform împărțirii sarcinilor.

Notă: Planificarea de mai jos urmează calendarul oficial al proiectului. Activitățile de preproducție desfășurate în săptămâna 7 sunt centralizate în documentația predată la începutul săptămânii 8.

Săptămâna 7 — Etapa de preproducție (predare la începutul săptămânii 8)

- **Membrul 1(Grigoraș Ioana-Daria):** definirea conceptului jocului, redactarea contextului, a gameplay-ului, a conținutului și a proiectării nivelurilor.
- **Membrul 2(Popescu Mircea):** realizarea diagramei UML, alegerea sprite-urilor și a resurselor grafice, descrierea interfeței cu utilizatorul, completarea bibliografiei și verificarea finală a documentului PDF.
- **Rezultat urmărit:** documentație completă, exportată în PDF și pregătită pentru încărcare la începutul săptămânii 8.

Săptămâna 8 — Meniul principal și nivelul 1

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea meniului principal și a navigării de bază.
- **Membrul 2(Popescu Mircea):** încărcarea hărții pentru nivelul 1 și configurarea primelor tile-uri.
- **Rezultat urmărit:** meniu funcțional și nivelul 1 încărcat.

Săptămâna 9 — Deplasarea personajului și camera

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea deplasării personajului principal pe hartă.
- **Membrul 2(Popescu Mircea):** implementarea camerei și adaptarea afișării în funcție de poziția jucătorului.
- **Rezultat urmărit:** personaj controlabil și cameră funcțională.

Săptămâna 10 — Nivelurile 2 și 3, inamici și NPC

- **Membrul 1(Grigoraș Ioana-Daria):** adăugarea nivelurilor 2 și 3, inclusiv obiectivele de progresie dintre niveluri.
- **Membrul 2(Popescu Mircea):** implementarea inamicilor și a NPC-urilor, precum și a interacțiunilor de bază cu acestea.
- **Rezultat urmărit:** toate cele trei niveluri disponibile, cu entități interactive.

Săptămâna 11 — Coliziuni

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea coliziunilor cu pereți, obstacole și zone blocate.
- **Membrul 2(Popescu Mircea):** testarea și reglarea coliziunilor pentru player, inamici și obiecte interactive.
- **Rezultat urmărit:** mișcare corectă și blocarea traversării elementelor solide.

Săptămâna 12 — SAVE / LOAD și baza de date

- **Membrul 1(Grigoraș Ioana-Daria):** implementarea sistemului de salvare și încărcare a progresului.
- **Membrul 2(Popescu Mircea):** integrarea bazei de date pentru progres, scoruri și best players.
- **Rezultat urmărit:** jocul poate salva și încărca progresul din baza de date.

Săptămâna 13 — Polish, patterns, excepții și bonusuri

- **Membrul 1(Grigoraș Ioana-Daria):** verificarea și integrarea șabloanelor de proiectare în implementare, tratarea excepțiilor și curățarea codului.

- **Membrul 2(Popescu Mircea):** finisarea interfeței, adăugarea de NPC suplimentari dacă este cazul și implementarea unuia sau mai multor bonusuri, precum fog of war, achievements sau ending secret.
- **Rezultat urmărit:** joc stabil, coerent și pregătit pentru prezentare.

Săptămâna 14 — Prezentarea finală

- **Membrul 1(Grigoraș Ioana-Daria):** pregătirea demonstrației tehnice și prezentarea implementării.
- **Membrul 2(Popescu Mircea):** pregătirea explicațiilor despre design, niveluri, UI, bază de date și bonusuri.
- **Rezultat urmărit:** prezentare finală completă a proiectului.

13. Explicația diagramei UML

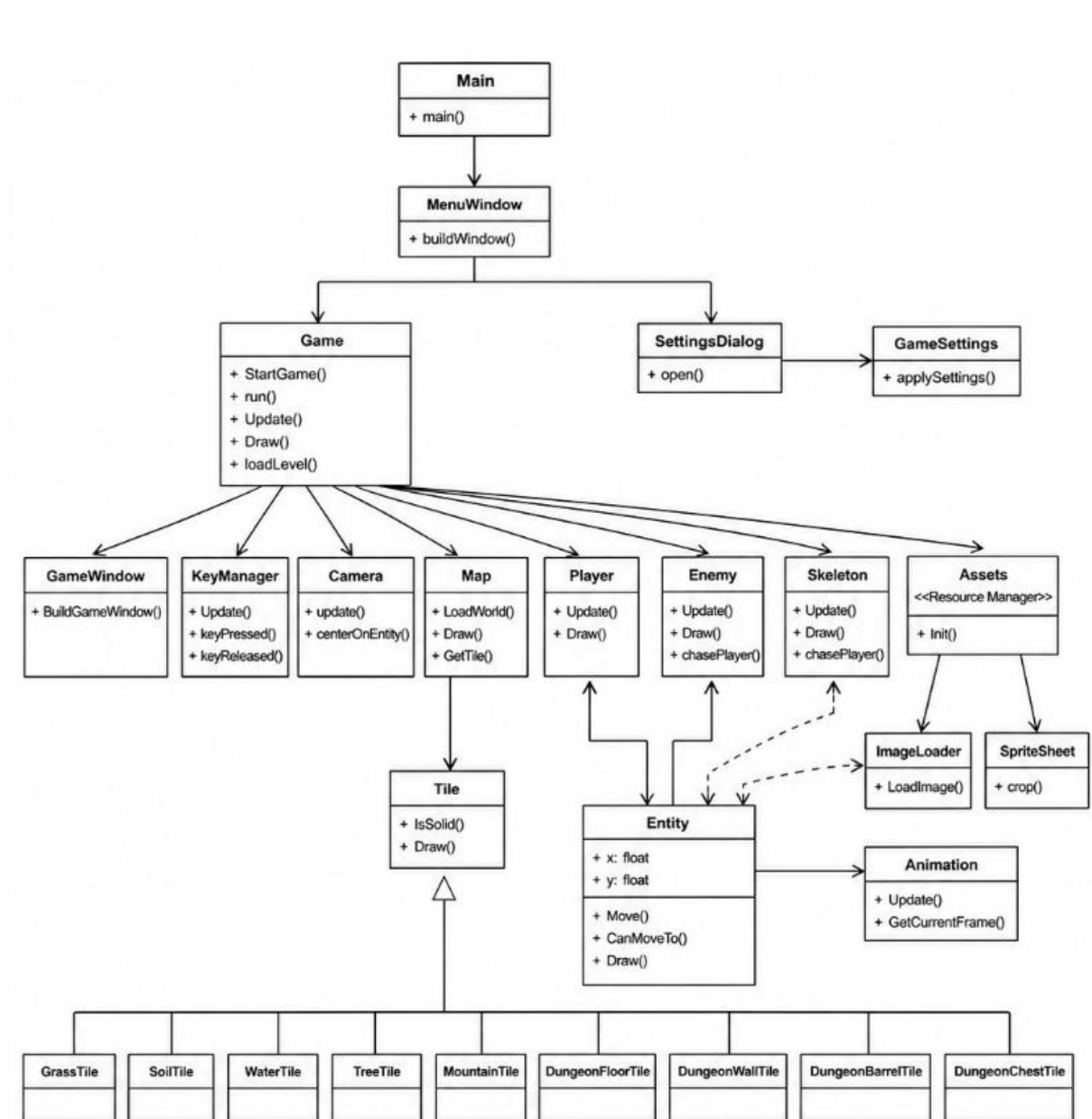


Diagrama UML de clase prezintă structura actuală a jocului *Aethelgard: Rise of the Fallen*

Crown, conform implementării curente. Clasa Main reprezintă punctul de intrare în aplicație și creează meniul principal prin clasa MenuWindow. Din meniul principal, utilizatorul poate porni jocul, poate accesa ecranul de setări sau poate închide aplicația.

Clasa MenuWindow creează o instanță a clasei Game atunci când utilizatorul apasă butonul de pornire a jocului. Clasa Game reprezintă nucleul aplicației și gestionează bucla principală a jocului prin metodele StartGame(), run(), Update() și Draw(). În această clasă sunt coordonate încărcarea nivelurilor, actualizarea inputului, actualizarea personajului, actualizarea inamicilor, poziționarea camerei, tranzițiile dintre niveluri și randarea grafică.

Fereastra de joc este gestionată prin clasa GameWindow, care creează structura grafică necesară pentru afișarea jocului, inclusiv obiectul de tip Canvas. Inputul de la tastatură este gestionat prin clasa KeyManager, care implementează interfața KeyListener și reține starea tastelor folosite pentru deplasare, pauză, selectarea opțiunilor din meniul de pauză și activarea modului debug.

Harta jocului este reprezentată prin clasa Map, care încarcă nivelurile din fișiere text externe și le transformă într-o matrice de identificatori de tile-uri. Fiecare valoare numerică din matrice corespunde unui obiect de tip Tile. Clasa Tile reprezintă baza pentru toate dalele jocului, iar clase precum GrassTile, SoilTile, WaterTile, TreeTile, MountainTile, DungeonFloorTile, DungeonWallTile, DungeonBarrelTile și DungeonChestTile definesc tipuri concrete de tile-uri. Unele dintre acestea suprascriu metoda IsSolid(), pentru a marca obstacolele care produc coliziuni.

Entitățile dinamice ale jocului sunt modelate prin superclasa abstractă Entity, care conține poziția, dimensiunea, viteza și metoda de verificare a coliziunilor. Din această clasă derivă Player, Enemy și Skeleton. Clasa Player reprezintă cavalerul controlat de utilizator și folosește obiecte de tip Animation pentru animațiile de repaus, alergare și întoarcere. Clasele Enemy și Skeleton reprezintă inamicii implementați în nivelurile curente și folosesc o logică simplă de urmărire a jucătorului într-o anumită rază de detecție.

Clasa Camera urmărește poziția jucătorului și limitează afișarea la marginile hărții, astfel încât să fie randată doar zona relevantă din lumea jocului. Pentru resursele grafice, proiectul utilizează clasa Assets, care funcționează ca un manager static de resurse. Aceasta folosește clasele ImageLoader și SpriteSheet pentru încărcarea imaginilor și decuparea sprite-urilor sau tile-urilor necesare jocului.

În etapa actuală, clasele pentru salvare, bază de date, scor, NPC și factory pentru inamici nu sunt încă implementate ca structuri separate. Acestea rămân direcții de dezvoltare pentru etapele următoare, în special pentru funcționalitățile de save/load, scor, bază de date, NPC-uri și extinderea sistemului de inamici.

14. Bibliografie

- [1] R. Nystrom, *Game Programming Patterns*.
- [2] OpenGameArt, "Game Art Assets."
- [3] Tiled Map Editor, "Official Website."
- [4] diagrams.net, "draw.io."
- [5] Aseprite, "Pixel Art Tool."
- [6] GIMP, "GNU Image Manipulation Program."

Resursele pentru asset-uri și editoare: OpenGameArt, PixelProspector, Aseprite, GIMP, Inkscape